

III. Method — Draft v2

Draft for the GS³LAM semantic-safety navigation paper (target: ICRA 2027). Written 2026-07-19, revised to v2 on 2026-07-20. Every figure and equation traces to ARCHITECTURE.md §2 or PROGRESS.md; the Stage 3 math is reproduced from ARCHITECTURE.md §2.3, where it was checked equation-for-equation against Tscholl, Nakka & Gunter (arXiv:2509.14421), and the equation numbers (Eq 8, 9a/9b, 13, 17) are confirmed against that paper. Method describes the system as designed and implemented; it does not report experimental findings. Attribution discipline and notation follow related_work_draft_v3.md, which this section sits beside. Judgment calls about the Method/Results boundary are flagged at the end. v2 changes: confidence-ellipsoid and QP-constraint equation numbers added (Eq 8, Eq 17); ALPHA_SCALE stated to its exact functional form; λ used for the covariance-inflation coefficient, matching ARCHITECTURE.md. Flags A/B/F/G stand as before; C/D/E/H are resolved.

Our system is a three-stage pipeline that turns monocular RGB video of an indoor scene into a semantics-aware navigation controller. Stage 1 reconstructs a semantically labeled 3D Gaussian Splatting map. Stage 2 assigns each mapped object a scalar safety score by querying a vision-language model once, on a single well-chosen view, and grafts that score onto the object's Gaussians as a new per-splat attribute. Stage 3 converts the safety-annotated map into a Control Barrier Function that filters a nominal controller's commands, weighting each splat's contribution to the safety constraint by its semantic score. Stages 1 and 2 run offline; only Stage 3 executes inside the control loop. Throughout, we distinguish components we build upon — GS³LAM's SLAM system, Gemini as the VLM, and Tscholl et al.'s collision-cone barrier — from this project's contribution, which is the semantic scoring, its grafting onto the map, and its integration into the barrier.

A. Stage 1 — Semantic Gaussian Splatting (GS³LAM)

The map front end is GS³LAM [Li et al., MM '24], used unmodified; the mechanism described in this subsection is GS³LAM's, and we claim none of it. GS³LAM runs three concurrent threads. A semantic thread applies DEVA — a SAM2-like video-object segmentation model — to produce a 2D semantic map for each frame. A tracking thread recovers the per-frame camera pose (a world-to-camera transform) by gradient descent. A mapping thread optimizes the Gaussian map by minimizing a combined photometric + geometric + semantic loss, in which DEVA's per-frame 2D semantic signal is one term optimized jointly with color and depth. Because the semantic term enters the mapping objective directly, splat boundaries are driven to align with object boundaries during optimization, which removes the separate post-hoc mask-projection / ray-casting step an earlier pipeline relied on (the full v1-to-v2 contrast is given in the design-history section and not repeated here).

Stage 1 emits two artifacts we consume downstream: a Gaussian map `gsplat.ply` carrying per-splat geometry, color, and opacity, and a parameter file `params.npz` holding the recovered camera trajectory and a 16-dimensional semantic vector (`obj_dc`) per splat. A separately trained lightweight decoder maps each 16-D semantic vector to a discrete object-class identifier, so that splats can be grouped by object. Everything in this stage is

GS³LAM's design; our pipeline begins by reading its outputs.

B. Stage 2 — Hero-Frame VLM Safety Rating

Stage 2 attaches a physical-safety judgment to each mapped object. Rather than query the VLM on every frame in which an object appears, we query it once, on the single frame that shows the object best — its *hero frame*.

Hero-frame selection. For a target object class, we project that object's splats into every recovered camera pose along the Stage 1 trajectory and measure the object's on-screen footprint as the bounding-box area of its projected points. The pose maximizing this footprint is selected as the hero frame. This reduces VLM invocations to one per object instead of one per keyframe, which is what removes the VLM from any per-frame cost.

Convex-hull background suppression. In the hero frame, we take the convex hull of the object's projected 2D points, rasterize it to a binary mask, and zero out all pixels outside it. The VLM therefore receives the isolated object rather than the surrounding scene, so its judgment reflects the object's own material and structure rather than incidental context.

VLM query. The masked crop is sent to Gemini — currently pinned to the generally-available gemini-3.5-flash model rather than a rolling alias, so the model reported in the write-up is reproducible across re-runs — with the response constrained to JSON, temperature 0.2, and the model's internal "thinking" pass disabled. The prompt frames the model as a physical safety auditor for a 3 kg wheeled robot (TurtleBot4) and requests a single float in $[0, 1]$, where 0.0 denotes a lethal hazard — fragile, easily tipped, or a cable — and 1.0 denotes an object that is completely safe to drive near or over.

Splat augmentation. The returned scalar is broadcast to every splat of the object and appended to the Gaussian map as a new safety field, using `numpy.lib.recfunctions`, producing an augmented `safety_gsplat.ply`. Splats belonging to objects that were not queried default to a safety of 0.0 — the most conservative value — and the downstream loader surfaces this default explicitly rather than treating a defaulted score as a genuine "lethal" judgment.

C. Stage 3 — Collision-Cone CBF and Semantic Weighting

Stage 3 consumes the safety-annotated map and produces a real-time safety filter: at each control step, a nominal command is modified as little as possible so that the robot stays outside every splat's collision cone, with each splat's contribution weighted by its semantic safety score. The geometric primitive is Tscholl et al.'s analytic collision-cone barrier, adopted unchanged; the semantic weighting layer on top of it is this project's contribution. We state the geometry first, exactly as verified in ARCHITECTURE.md §2.3, then the weighting.

Geometric collision-cone barrier (Tscholl, Nakka & Gunter, arXiv:2509.14421). Each splat is treated as a confidence ellipsoid. From the splat's rotation R and scales, its covariance is $\Sigma = R S S^T R^T$; we define $A := \Sigma^{-1}$ and the confidence ellipsoid (Eq 8)

$$\square = \{ \mathbf{x} : (\mathbf{x} - \boldsymbol{\mu})^T A (\mathbf{x} - \boldsymbol{\mu}) \leq c^2 \}, \quad c^2 = \chi^2_{(3, 0.99)},$$

with c^2 the 99% quantile of the chi-square distribution on 3 degrees of freedom. For a robot at position p with velocity v , let $r := \mu - p$ and define

$$\beta := v^T A v, \gamma := r^T A r - c^2, \delta := r^T A v.$$

A collision between the forward velocity ray and the ellipsoid exists iff (Eq 9a/9b)

$$\beta\gamma - \delta^2 \leq 0 \text{ and } \delta \geq 0.$$

The per-splat barrier function is (Eq 13)

$$h(p, v) := \beta\gamma - \delta^2,$$

so that $h \geq 0$ defines the splat's safe set. At each control step the filter solves the quadratic program

$$\min_u \|u - \bar{u}\|^2 \text{ s.t. } w(x)^T u \geq -(k_\alpha / 2) \cdot h(p, v) \text{ (Eq 17, for each active splat), } \|u\| \leq a_{\max},$$

where $\bar{u}$ is the nominal command, $a_{\max}$ the actuator bound, k_α the class-K gain (Tscholl et al.'s constant p_k , renamed k_α in our codebase to avoid clashing with the robot position p), and

$$w(x) := \gamma \cdot A v - \delta \cdot A r.$$

Robot extent is handled by Minkowski inflation (Remark 3): for a robot of radius ρ , c is replaced by $c_m = c + \rho / s_{\min}$, where $s_{\min}$ is the smallest splat scale; because R is orthogonal, $s_{\min}$ is a plain minimum over the splat's scales and needs no SVD.

Semantic weighting. The safety scalar from Stage 2 enters this geometric filter through one of two swappable mechanisms, both defined over a shared NONE baseline that never reads the safety field and reproduces the pure-geometric filter above. The two mechanisms act on different axes of the control response, which is why both are implemented and both are subjected to the experimental comparison.

- **ALPHA_SCALE — class-K gain modulation.** The per-splat class-K gain is set to $k_{\alpha,i} = k_{\alpha_base} \cdot \max(f(\text{safety}_i), f_{\min})$, where f is a monotonic map of the safety score. The floor $f_{\min} = 1e-3$ is applied to the multiplier factor $f(\text{safety}_i)$ — inside the $\max$, before the k_{α_base} scaling — so the effective minimum gain is $k_{\alpha_base} \cdot f_{\min}$ and the constraint cannot collapse to exactly zero (which would silently disable the barrier for that splat). The implemented default is the identity, $f(s) = s$, giving $k_{\alpha,i} = k_{\alpha_base} \cdot \max(\text{safety}_i, f_{\min})$: the safety score acts as a direct multiplier on the gain — higher safety, larger gain — not an inverse. A lower score therefore yields a smaller gain, and since the QP constraint reads $w(x)^T u \geq -(k_\alpha / 2) \cdot h$, a smaller gain enforces the barrier more conservatively. This changes how aggressively the robot decelerates as it approaches an object — its longitudinal approach speed — while leaving the geometric region defined by the barrier unchanged. The map f is itself an open design parameter, not settled by the implementation; the identity is only the current default.

- **COV_INFLATE — covariance inflation.** The splat's effective covariance is scaled as $\Sigma_{\text{eff}} = \Sigma \cdot (1 + \lambda (1 - \text{safety}))$, where λ is an inflation coefficient; in implementation this is a scalar operation on A and $s_{\min}$, avoiding any rebuild or re-inversion of Σ . A

lower-safety object therefore presents a geometrically larger ellipsoid, and because the collision cone is computed from the inflated A, the barrier routes the robot laterally around a hazardous object rather than merely slowing for it. The inflation coefficient λ (the codebase's `cov_inflate_gamma`) is a free parameter of the method: the CBF configuration supplies a default of 1.0, marked in-code as a value to sweep rather than a tuned constant, so its selection is part of the experimental evaluation rather than a fixed design commitment.

The two mechanisms encode different interpretations of "this object warrants more caution": `ALPHA_SCALE` expresses caution as reduced approach speed at unchanged clearance, while `COV_INFLATE` expresses it as increased lateral clearance. Which better matches the intended navigation behavior is the open question the experiments are designed to answer; implementing both as swappable modes over one shared geometric baseline makes that comparison controlled.

Solver. The quadratic program is solved by default with Clarabel, an interior-point conic solver — the choice Tscholl et al. make as well — because it represents the actuator bound $\|u\| \leq a_{\max}$ natively as a second-order cone constraint rather than approximating it with linear or per-axis box constraints. An SLSQP backend is retained as a selectable fallback behind a shared solver interface, so the filter is not tied to a single QP implementation.

Contribution boundary. The collision-cone construction — the confidence ellipsoid, the existence condition, the barrier h , the QP, and the Minkowski inflation — is Tscholl et al.'s, used here without modification. This project's Stage 3 contribution is the semantic weighting layer: the two mechanisms by which a VLM-derived per-splat safety score modulates that geometric barrier, and their integration into a single swappable filter over a shared baseline. The two are orthogonal in exactly the sense established in the Related Work — the collision cone determines *how* a splat constrains the robot's motion; the safety score determines *how much*, as a function of what the object is.

Flags for Aashrut — Method/Results boundary and notation calls (not part of the paper)

Flag A — I did not follow GS3LAM_PAPER_SCOPE.md's "unvalidated alternatives" wording, per your instruction. Cross-checked against PROGRESS.md: the two modes are real, implemented, and swappable over a shared NONE baseline (PROGRESS.md lines 27–30, 58–64), and the COV_INFLATE behavior that the old wording hedged around has since been disambiguated. I frame both modes as *mechanisms under experimental comparison* — what each does and why both are worth testing — with no claim that either is decided or validated. This matches the current code state, not the pre-disambiguation guidance.

Flag B — Method/Results boundary: what I deliberately kept OUT. Per your instruction, none of the following appears in this section, because each is a finding, not a mechanism: the synthetic sanity-check outcome (ALPHA_SCALE reaching the goal with a slower approach along the same path); the COV_INFLATE fixed-gain disambiguation and its $\sim 12\times \rightarrow \sim 5.9\times$ correction; and the SLSQP-suboptimality result that motivated the Clarabel default. Clarabel-as-default is stated only as a design decision (native second-order-cone handling of the actuator bound), which is its Method-appropriate justification. If you'd rather any of these live in Method, tell me — but I read the boundary as: Method says what the system does and why it was built this way, Results says what was found.

Flag C — Equation numbering: resolved. The ellipsoid is now labeled (Eq 8) and the QP safety constraint (Eq 17), both confirmed correct against the actual Tscholl et al. paper (per your verification), alongside the existence condition (Eq 9a/9b) and barrier (Eq 13) that ARCHITECTURE.md already carried. The Minkowski inflation remains unnumbered — it is Remark 3 in the source, not a numbered equation, so I left it as a remark reference rather than assigning it an equation number.

Flag D — Symbol clash: resolved. The inflation coefficient is now λ in both this draft and ARCHITECTURE.md §2.3 (updated 2026-07-20, with a parenthetical noting it is distinct from the collision-cone $\gamma := \mathbf{r}^T \mathbf{A} \mathbf{r} - c^2$). The codebase variable stays `cov_inflate_gamma` — a rename there wasn't in scope and would touch working code. If you'd prefer a different final symbol than λ , change both docs together.

Flag E — resolved (investigated, and one real bug fixed). `git blame` confirmed the top-5 cap (`np.argsort(-counts)[:5]`) is first-pass demo-driver scaffolding from the original 2026-04-20 commit, never a recorded scope decision: it lives only in `__main__`, every library function is per-object, and it contradicts the documented per-object method. Left in the code as-is (removing it belongs with a real Stage 1/2 run, not this pass) but excluded from the paper, which correctly describes per-object behavior. Investigating it surfaced a separate, genuinely load-bearing bug: queried objects whose call *failed* defaulted to a safety of 1.0 ("completely safe"), an inverted fail-safe versus the 0.0 default used for unqueried objects — so a VLM timeout on a real hazard silently told the CBF to drive over it. Corrected to 0.0 (uniformly conservative "no valid score") and recorded in PROGRESS.md (2026-07-20) per CLAUDE.md's rule on safety-score-scale changes. The Stage 2 method text is unaffected by either.

Flag F — Real-data status kept out of Method, deliberately. PROGRESS.md is explicit (lines 76, 126, 203–205) that no real gsplat.ply / params.npz / classifier.pth / safety_gsplat.ply exists yet, and that Stage 3 has only ever run on a synthetic scene. Per your instruction I describe Stages 1–2 as *designed and implemented* and did not claim any real output. I also did not insert a "not yet validated" disclaimer into Method itself, since that's a Status/Results fact — but if you want a one-line honest pointer in Method (e.g., a forward reference to the experimental-status section), say so and I'll add it.

Flag G — The unqueried-splat default (safety = 0.0). I included this as a one-sentence design fact in Stage 2 because it genuinely shapes how Stage 3 reads the map. It could instead be deferred to Discussion as a limitation (it is ambiguous with a real "lethal" score and unrecoverable from the PLY alone, per ARCHITECTURE.md's data-contract note). Your call on placement.

Flag H — resolved. Pulled the exact form from semantic_weighting.alpha_gain_per_splat: $k_{\alpha,i} = k_{\alpha_base} \cdot \max(f(\text{safety}_i), f_{min})$, with default $f = \text{identity}$ and $f_{min} = 1e-3$ — the safety score is a *direct* multiplier on the gain (not an inverse), now stated exactly in the ALPHA_SCALE paragraph. The direction is confirmed against the QP constraint (smaller gain → more conservative), matching the earlier plain-language claim. The one open item the code itself flags is that f is a configurable monotonic map (identity is only the default), which the paragraph now notes rather than presenting the identity as settled.

Flag I — COV_INFLATE λ : corrected a factual error, and a Method/Results guard. The earlier wording ("no built-in default... a parameter to be swept, e.g. {0.5, 1.0, 2.0}") was wrong on both counts. Verified against the code: CBFQPConfig.cov_inflate_gamma defaults to **1.0** (qp_filter.py:59, commented "a value to sweep, not a settled constant"), and eval_cbf_modes.py's --cov-gamma also defaults to 1.0 — only the low-level inflate_covariance() helper lacks a default, which is not the caller-facing config. The {0.5, 1.0, 2.0} sweep was invented and has never run. Per PROGRESS.md, the only λ values ever exercised are **{0.3, 1.0}**, all on the synthetic scene: the disambiguation's headline work (the k_alpha_base sweep, the wide-corridor comparison, the $\sim 11.91\times \rightarrow 5.9\times$ Clarabel result, the regression tests) fixed $\lambda = 1.0$, with a single 0.3 comparison point in the Step-0 table (itself later flagged in PROGRESS as measured against the wrong ellipsoid). I therefore rewrote the body sentence to state only the design fact (config-default 1.0, marked in-code as a sweep target, selection deferred to evaluation) and deliberately did **not** name a sweep set, so Method sets up no multi- γ sweep that Results has not delivered. If you do plan a specific λ sweep for the real-scene experiments, that belongs in the Experimental Protocol section, stated as planned, not implied here.